

DevOps Engineer

1st Round Interview Preparation Guide

20 Questions Asked Right Now — With Expert Answers

CI/CD	Docker Internals	Kubernetes	Terraform
Networking	Observability	Security	Incident Response

Sanket Ajay Chopade

DevOps Engineer | Pune, India

Table of Contents

CI/CD & PIPELINES

Q01 How does a CI/CD pipeline work end-to-end (commit → production)?

CONTAINERS & DOCKER

Q02 What happens internally when you run a Docker container?

KUBERNETES

Q03 How does Kubernetes schedule pods on nodes?

Q04 What causes CrashLoopBackOff and how do you debug it?

Q05 Difference between readiness and liveness probes?

Q06 Explain Kubernetes architecture (API server, scheduler, etcd).

Q07 How does service discovery work in Kubernetes?

Q08 What happens when you run kubectl apply internally?

Q09 Difference between Deployment, StatefulSet, and DaemonSet?

CONFIGURATION & INFRA

Q10 How do you handle configuration management across environments?

TERRAFORM & IaC

Q11 How does Terraform manage state and dependencies internally?

Q12 What is Terraform drift and how do you detect it?

Q13 How do you design Infrastructure as Code for multiple environments?

NETWORKING

Q14 How does DNS resolution work step by step?

Q15 What is the difference between Layer 4 and Layer 7 load balancers?

OBSERVABILITY

Q16 If your application is slow under load, how do you identify bottlenecks?

Q17 How do you design monitoring and alerting for production systems?

SECURITY

Q18 How do you implement secure authentication between services?

DEPLOYMENT STRATEGIES

Q19 How do you design zero-downtime deployment strategies?

INCIDENT RESPONSE

Q20 If a production system goes down, how do you approach debugging step by step?

CI/CD & PIPELINES

CI/CD & PIPELINES

Q01

How does a CI/CD pipeline work end-to-end (commit → production)?

■ Pipeline Stages

- **Code Commit:** Developer pushes to a branch (e.g., feature/xyz); webhook triggers the pipeline.
- **Source Stage:** Pipeline fetches source from VCS (GitHub, GitLab, Bitbucket).
- **Build Stage:** Code is compiled/packaged. Docker image built and tagged (e.g., commit SHA).
- **Test Stage:** Unit tests, integration tests, SAST (SonarQube), container scan (Trivy). Fail fast — pipeline aborts on failure.
- **Artifact Stage:** Image pushed to registry (DockerHub / ECR). Artifacts stored in S3 or Nexus.
- **Deploy to Dev/Staging:** Helm/Kustomize applies manifests to cluster. Automated smoke tests run.
- **Approval Gate:** Optional manual approval for production (in Jenkins: input step; GitHub Actions: environment protection rules).
- **Deploy to Production:** Rolling / Blue-Green / Canary strategy executes. Health checks verified.
- **Post-Deploy:** Monitoring dashboards checked (Datadog/Grafana). Alerts configured for SLO breaches.

■ Key Principles

- **Fail fast:** test early so bugs never reach production.
- **Immutable artifacts:** same image deployed to dev, staging, and prod — never rebuild.
- **GitOps optional:** ArgoCD/Flux watches Git repo and syncs cluster state automatically.

■ Common Tools

- **CI:** Jenkins, GitHub Actions, GitLab CI, CircleCI
- **Registry:** ECR, DockerHub, GCR
- **CD:** ArgoCD, Flux, Spinnaker, Helm

CONTAINERS & DOCKER

What happens internally when you run a Docker container?

■ Step-by-Step Internals

`docker run nginx` — Docker CLI sends request to Docker Daemon (`dockerd`) via REST API (Unix socket).

- **Image Pull:** Daemon checks local image cache. If missing, pulls layers from registry (content-addressed by SHA256).
- **Layer Overlay:** Docker uses OverlayFS (overlay2 driver) — stacks read-only image layers + a writable container layer on top.
- **Namespace Isolation:** Linux kernel creates namespaces — PID (process isolation), NET (network stack), MNT (filesystem), UTS (hostname), IPC, USER.
- **cgroups:** Linux control groups enforce resource limits — CPU shares, memory limits, I/O throttling.
- **Network Setup:** Docker creates a veth pair; one end in container, one in `docker0` bridge. Container gets its own IP.
- **Entry Point Execution:** Container process (PID 1 inside namespace) starts — e.g., `nginx` master process.
- **Union Filesystem:** All writes go to the writable layer only. Image layers remain untouched.

■ What Makes Containers Lightweight

- Containers share the host kernel — no hypervisor, no guest OS boot.
- Start in milliseconds vs minutes for VMs.
- Isolation is namespaces + cgroups, NOT full hardware virtualization.

KUBERNETES

How does Kubernetes schedule pods on nodes?

■ Scheduling Pipeline

- **Filtering (Predicates):** Scheduler eliminates nodes that cannot run the pod. Checks: resource requests (CPU/memory), nodeSelector, nodeName, taints/tolerations, affinity rules, PodDisruptionBudgets, port conflicts.
- **Scoring (Priorities):** Remaining nodes scored 0–100. Key scoring functions: LeastRequestedPriority (prefer least loaded), BalancedResourceAllocation, NodeAffinityPriority, ImageLocalityPriority (node already has image).
- **Binding:** Highest-scoring node selected. Scheduler writes binding to API server. Kubelet on that node picks it up and starts the pod.

■ Influencing Scheduling

- **nodeSelector:** Simple label match — schedule on nodes with disk=ssd.
- **Node Affinity:** Richer rules — requiredDuringSchedulingIgnoredDuringExecution (hard) vs preferredDuringScheduling (soft).
- **Taints & Tolerations:** Taint a node (e.g., gpu=true:NoSchedule); only pods with matching toleration land there.
- **Pod Affinity/Anti-Affinity:** Co-locate or spread pods across topology zones.
- **Resource Requests/Limits:** Pod declares CPU/memory needs; scheduler uses requests (not limits) for placement.

■ Important Edge Cases

- Scheduler is pluggable — custom schedulers possible.
- DaemonSet pods bypass normal scheduling (use NodeAffinity internally).
- Pending pod = no suitable node found — check kubectl describe pod for reason.

What causes CrashLoopBackOff and how do you debug it?

■ What It Means

- Container starts, crashes, Kubernetes restarts it, it crashes again. Backoff interval increases exponentially (10s → 20s → 40s... max 5 min).
- It is a symptom, not a root cause — always dig deeper.

■ Common Root Causes

- **Application error:** Unhandled exception, missing env variable, config file not found.
- **Wrong entrypoint/command:** Container exits immediately (exit code 0 or non-zero).
- **OOMKilled:** Container exceeds memory limit — exit code 137.
- **Liveness probe failure:** Probe misconfigured or app not ready in time.
- **Missing secrets/configmaps:** App panics on startup when required config is absent.
- **Port conflicts or permission issues:** App cannot bind to required port.

■ Debugging Steps

`kubectl describe pod` – Check Events section for OOMKilled, probe failures, image pull errors.

`kubectl logs --previous` – Logs from the PREVIOUS (crashed) container instance.

`kubectl logs -c` – For multi-container pods.

`kubectl exec -it -- /bin/sh` – If container is briefly running, exec in to inspect.

- **Check exit codes:** 0=clean exit, 1=app error, 137=OOMKilled, 139=segfault.
- Temporarily override entrypoint with sleep infinity to keep it running for inspection.

Difference between readiness and liveness probes?

■ Liveness Probe

- **Purpose:** Is the container still alive? Should Kubernetes restart it?
- **Action on failure:** Container is KILLED and restarted.
- **Use case:** Detect deadlocks or corrupted state where the app is running but non-functional.
- **Example:** HTTP GET /healthz — if it returns non-2xx, restart.

■ Readiness Probe

- **Purpose:** Is the container ready to serve traffic?
- **Action on failure:** Pod is REMOVED from Service endpoints — no traffic sent, but container NOT restarted.
- **Use case:** App needs time to warm up (load cache, connect to DB). During rolling updates, ensure traffic only goes to healthy pods.
- **Example:** HTTP GET /ready — returns 503 until DB connection established.

■ Startup Probe (Bonus)

- Delays liveness/readiness checks until initial startup completes.
- Prevents CrashLoopBackOff on slow-starting legacy apps.

■ Critical Distinction

- Wrong liveness probe = cascading restarts under load (probe times out, app killed needlessly).
- Missing readiness probe = traffic hits pods that are starting up, causing 502/503 errors.
- **Rule of thumb:** liveness threshold should be LOOSER than readiness.

Explain Kubernetes architecture (API server, scheduler, etcd).

■ Control Plane Components

- **API Server (kube-apiserver):** The single entry point for all cluster operations. Validates and persists state to etcd. All components communicate ONLY through the API server. Exposes REST API — kubectl, controllers, and external tools all use it.
- **etcd:** Distributed key-value store. Single source of truth for ALL cluster state (pod specs, node info, secrets, configmaps). Uses Raft consensus for HA. If etcd is lost, cluster is unrecoverable without backup.
- **Scheduler (kube-scheduler):** Watches for unscheduled pods (spec.nodeName is empty). Runs filtering + scoring to assign pods to nodes. Writes the binding back to API server.
- **Controller Manager (kube-controller-manager):** Runs reconciliation loops. ReplicaSet controller ensures desired pod count. Node controller handles node failures. Job controller manages batch workloads.
- **Cloud Controller Manager:** Integrates with cloud provider APIs (AWS, GCP, Azure) for load balancers, node provisioning, storage.

■ Worker Node Components

- **Kubelet:** Agent on every node. Reads pod specs from API server. Creates/manages containers via container runtime (containerd/CRI-O). Reports node and pod status back.
- **Kube-proxy:** Manages network rules (iptables/IPVS) for Service routing. Ensures traffic to a Service IP gets load-balanced across pod endpoints.
- **Container Runtime:** containerd (most common), CRI-O. Implements CRI (Container Runtime Interface). Actually pulls images and runs containers.

■ Key Design Principle

- All components are stateless and watch/reconcile — they keep comparing desired state (etcd) vs actual state and take corrective action.

How does service discovery work in Kubernetes?

■ CoreDNS-Based Discovery

- **Every Service gets a DNS name:** ..svc.cluster.local
- CoreDNS runs as a pod in kube-system and watches the API server for Service changes.
- Pods have /etc/resolv.conf pointing to CoreDNS ClusterIP — all DNS queries route through it.
- **Example:** frontend pod queries backend-service.default.svc.cluster.local → CoreDNS returns ClusterIP.

■ How Services Route Traffic

- **ClusterIP Service:** Virtual IP (VIP) assigned. kube-proxy sets up iptables/IPVS rules to DNAT traffic to one of the healthy pod IPs.
- **Endpoints object:** Automatically updated with IPs of ready pods matching the Service selector. Unhealthy pods (failed readiness probe) removed from Endpoints.
- **Headless Service (clusterIP: None):** DNS returns individual pod IPs directly — used for StatefulSets where clients need to connect to specific pods.

■ Environment Variables (Legacy)

- Kubernetes also injects SERVICE_HOST and SERVICE_PORT env vars into pods, but DNS is preferred and more dynamic.

What happens when you run `kubectl apply` internally?

■ Step-by-Step Flow

`kubectl` reads the YAML manifest and serializes it to JSON.

- Sends HTTP PATCH (strategic merge) or POST request to API server with the manifest.
- **API Server:** Authenticates (cert/token), authorizes (RBAC check), runs admission controllers (webhooks, PodSecurityAdmission, ResourceQuota).
- API Server validates the object schema against OpenAPI spec.
- Object written to etcd. The last-applied-configuration annotation is updated (this is what enables 3-way merge diff).
- Relevant controller (e.g., Deployment controller) detects the change via watch.
- **Controller reconciles:** creates/updates ReplicaSets, which create Pods.
- Scheduler assigns pods to nodes. Kubelet creates containers.

■ apply vs create vs replace

`kubectl create`: Fails if resource already exists.

`kubectl apply`: 3-way merge (current state, last-applied, new desired) – safe for repeated use.

`kubectl replace`: Full replacement – deletes and recreates. Destructive.

■ Admission Controllers Matter

- MutatingAdmissionWebhook can inject sidecars (e.g., Istio envoy), add labels, set defaults.
- ValidatingAdmissionWebhook can reject objects that violate policy (e.g., OPA/Gatekeeper).

Difference between Deployment, StatefulSet, and DaemonSet?

■ Deployment

- Manages stateless applications. Pods are interchangeable — any pod can serve any request.
- Supports rolling updates and rollbacks (kubectl rollout undo).
- Pods get random names (nginx-abc12-xyz). No persistent identity.
- **Use for:** web servers, APIs, workers — anything stateless.
- **Scaling is easy:** kubectl scale deployment nginx --replicas=5

■ StatefulSet

- Manages stateful applications where identity matters.
- **Pods get stable, ordered names:** mysql-0, mysql-1, mysql-2.
- Each pod gets its own PersistentVolumeClaim — survives pod restarts.
- Ordered startup and shutdown (mysql-0 starts before mysql-1).
- **Stable DNS:** mysql-0.mysql-headless.default.svc.cluster.local
- **Use for:** databases (MySQL, MongoDB, Kafka, Zookeeper, Elasticsearch).

■ DaemonSet

- Ensures exactly ONE pod runs on every node (or a subset via nodeSelector).
- Automatically adds pod to new nodes as they join the cluster.
- **Use for:** log collectors (Fluentd), monitoring agents (Datadog Agent, Prometheus Node Exporter), network plugins (Calico, Cilium), security agents.
- Bypasses normal scheduler — placed on nodes directly.

CONFIGURATION & INFRA

How do you handle configuration management across environments?

■ Core Strategy

- Separate config from code (12-factor app principle).
- Never hardcode environment-specific values in application code or Docker images.

■ Kubernetes-Native Approach

- **ConfigMaps**: Non-sensitive config (app settings, feature flags). Mounted as env vars or volume files.
- **Secrets**: Sensitive data (passwords, API keys). Base64-encoded (NOT encrypted by default — enable etcd encryption at rest).
- **Kustomize**: Overlay system — base/ has common config, overlays/dev|staging|prod/ patch environment-specific values. No templating language needed.
- **Helm**: Charts with values.yaml per environment (values-dev.yaml, values-prod.yaml). Templated YAML.

■ External Secrets Management

- **External Secrets Operator (ESO)**: Syncs secrets from AWS Secrets Manager, Vault, GCP Secret Manager into Kubernetes Secrets automatically.
- **AWS Secrets Manager + IRSA**: Pods assume IAM roles via OIDC — no static credentials.
- **HashiCorp Vault**: Agent sidecar injects secrets into pod filesystem at runtime.

■ Environment Promotion Flow

- Dev config → tested → promoted to staging → approved → promoted to prod.
- **GitOps**: Config changes committed to Git, ArgoCD syncs cluster automatically.

TERRAFORM & IaC

How does Terraform manage state and dependencies internally?

■ State File

`terraform.tfstate`: JSON file mapping your HCL config to real infrastructure resources (AWS IDs, ARNs, IPs).

- **On every apply**: Terraform reads state, computes diff against desired config, plans changes, applies, updates state.
- **Remote State**: Store in S3 + DynamoDB (locking) or Terraform Cloud. Prevents state corruption from concurrent runs.
- **State locking**: DynamoDB table prevents two engineers running apply simultaneously.

■ Dependency Graph

- Terraform builds a DAG (Directed Acyclic Graph) of all resources.
- **Explicit dependency**: `depends_on = [aws_vpc.main]` — forces ordering.
- **Implicit dependency**: `aws_subnet.main` references `aws_vpc.main.id` — Terraform auto-detects dependency.
- Independent resources run in PARALLEL (up to 10 by default). Dependent resources run in sequence.

■ State Commands

`terraform state list` — show all tracked resources.

`terraform state mv` — move resource without destroying (e.g., during refactor).

`terraform state rm` — remove from state without destroying real resource.

`terraform import` — bring existing infra under Terraform management.

■ Important Warning

- Never manually edit `terraform.tfstate`. Never store state in Git — it contains plaintext secrets.

What is Terraform drift and how do you detect it?

■ What Is Drift

- Drift = divergence between Terraform state (desired) and actual cloud infrastructure (real).
- **Causes:** Manual changes via AWS Console, CLI, or other tools. Autoscaling changes. External processes modifying infrastructure.

■ Detection Methods

`terraform plan`: Compares state against real infra by calling cloud provider APIs. Shows what would change. Run as a scheduled check in CI (plan-only pipeline).

`terraform refresh`: Updates state to match real infra. Deprecated in newer versions – use `terraform apply -refresh-only` instead.

- **AWS Config + Config Rules:** Detects unauthorized changes to AWS resources independently.
- **Driftctl (open-source tool):** Comprehensive drift detection across all cloud resources, including ones not in Terraform state (unmanaged resources).

■ Remediation Strategy

- **Option 1 — Revert:** Re-apply Terraform to restore desired state (automated, preferred).
- **Option 2 — Absorb:** If change was intentional, update Terraform config to match and commit.
- **Option 3 — Import:** If new resource was created manually, use `terraform import` to bring it under management.
- **Prevention:** Use SCPs/IAM policies to restrict direct console changes. Enforce all changes via Terraform PRs.

How do you design Infrastructure as Code for multiple environments?

■ Pattern 1 — Workspaces (Simple)

`terraform workspace new staging` – separate state per workspace.

- **Limitations:** Same config for all envs — hard to have different instance sizes or VPC CIDRs. Not recommended for large teams.

■ Pattern 2 — Directory-per-Environment (Recommended)

Structure: `environments/dev/`, `environments/staging/`, `environments/prod/`

- Each directory has its own backend config and tfvars file.
- **Pros:** Full isolation, different configs per env, safe blast radius.
- **Cons:** Code duplication — solved by using shared modules.

■ Pattern 3 — Modules (Best Practice)

`modules/vpc/`, `modules/eks/`, `modules/rds/` – reusable infrastructure components.

`environments/prod/main.tf` calls modules with prod-specific variables.

- Single source of truth for infra logic. Environment differences are just variable values.

■ Recommended Folder Structure

`modules/` – reusable components

`environments/dev/main.tf` + `terraform.tfvars`

`environments/staging/main.tf` + `terraform.tfvars`

`environments/prod/main.tf` + `terraform.tfvars`

- Each env has separate remote state backend (separate S3 bucket or prefix).

■ CI/CD Integration

- PRs run terraform plan. Merges to main trigger terraform apply.
- Prod apply requires manual approval gate. State stored in separate S3 buckets per env.

NETWORKING

How does DNS resolution work step by step?

■ Full Resolution Flow

- Browser/app requests `www.example.com` — checks local browser cache first.
- **OS DNS Cache:** Checks `/etc/hosts` and local resolver cache.
- **Recursive Resolver (your ISP or 8.8.8.8):** If not cached, resolver starts the recursive lookup.
- **Root Name Server:** Resolver queries root (.): 'Who handles .com?' → returns TLD server IPs.
- **TLD Name Server:** Resolver queries .com TLD server: 'Who handles example.com?' → returns authoritative NS.
- **Authoritative Name Server:** Resolver queries example.com's name server. Returns the actual A record (IP address).
- Response cached at each level with TTL. Browser connects to the IP.

■ Record Types

- **A:** Domain → IPv4 address
- **AAAA:** Domain → IPv6 address
- **CNAME:** Alias → another domain (cannot be used at apex/root domain)
- **MX:** Mail exchange servers
- **TXT:** Arbitrary text (SPF, DKIM, domain verification)
- **NS:** Name server delegation
- **SOA:** Start of authority — primary NS, admin email, serial/refresh/retry/expire

■ DevOps Relevance

- Low TTL during migrations — allows quick failover.
- **Route53 health checks + failover routing:** Automatically redirects traffic if primary endpoint fails.
- **Split-horizon DNS:** Internal and external different IPs for same domain.
- **Negative caching (NXDOMAIN TTL):** Missing records also cached — be careful during new domain setup.

What is the difference between Layer 4 and Layer 7 load balancers?

■ Layer 4 (Transport Layer)

- Operates on TCP/UDP. Makes routing decisions based on IP address and port ONLY.
- Cannot inspect HTTP headers, URL paths, or cookies.
- Very fast — minimal processing overhead. Connection-level load balancing.
- **AWS equivalent:** NLB (Network Load Balancer). Supports ultra-low latency, TCP passthrough.
- **Use cases:** Non-HTTP traffic (databases, SMTP, game servers), high-throughput scenarios.

■ Layer 7 (Application Layer)

- Operates on HTTP/HTTPS. Can inspect request content — URL path, headers, cookies, query params.
- **Enables content-based routing:** `/api/*` → API servers, `/static/*` → CDN/S3.
- **Supports:** SSL termination, sticky sessions (cookie-based), host-based routing (`app1.example.com` vs `app2.example.com`).
- **AWS equivalent:** ALB (Application Load Balancer). Kubernetes Ingress controllers (nginx, traefik, ALB Ingress Controller).
- **Use cases:** HTTP microservices, canary deployments (route 10% traffic to v2), A/B testing.

■ Comparison Summary

- **L4:** Faster, simpler, no content awareness, TCP/UDP only.
- **L7:** Smarter routing, SSL termination, more features, slightly more overhead.
- Kubernetes Ingress = L7 load balancer. Kubernetes Service (LoadBalancer type) = L4 by default.

OBSERVABILITY

If your application is slow under load, how do you identify bottlenecks?

■ Systematic Debugging Approach

- **1. Check golden signals first:** Latency (p50/p95/p99), Error rate, Throughput (RPS), Saturation.
- **2. Infrastructure layer:** CPU, memory, disk I/O, network — `kubectl top pods/nodes`, Datadog host map.
- **3. Application layer:** APM traces (Datadog APM, Jaeger) — find which service/function is slow.
- **4. Database layer:** Slow query logs, connection pool exhaustion, lock contention, missing indexes.
- **5. Network layer:** DNS resolution times, connection timeouts, inter-service latency.

■ Common Bottlenecks

- **CPU throttling:** Pod hitting CPU limits — requests and limits not set correctly.
- **Memory pressure:** OOMKilled pods, JVM GC pauses, too-small heap.
- **Thread/connection pool exhaustion:** App can't open new DB connections or HTTP connections.
- **N+1 query problem:** ORM making 100 queries where 1 join would suffice.
- **Lock contention:** Database row locks causing serialization.
- **HPA not scaling:** Missing metrics, minReplicas too low, scale-up delay too high.

■ Tools & Techniques

```
kubectl top pods --sort-by=cpu
```

- **Distributed tracing:** Find the slow span — is it your service, a DB call, or a downstream API?
- **Load testing:** k6, Locust, wrk — reproduce the problem under controlled conditions.
- **Profiling:** pprof (Go), py-spy (Python), async-profiler (Java) — identify hot code paths.
- **Flame graphs:** Visual CPU time breakdown.

How do you design monitoring and alerting for production systems?

■ The Three Pillars

- **Metrics:** Numeric time-series data. Prometheus scrapes /metrics endpoints. Grafana dashboards visualize.
- **Logs:** Structured (JSON) logs shipped via Fluentd/Fluent Bit to Elasticsearch or CloudWatch.
- **Traces:** Distributed request traces (Jaeger, Datadog APM, AWS X-Ray). Correlate across services.

■ What to Monitor (4 Golden Signals)

- **Latency:** Request duration (p50, p95, p99). Alert on p99 > SLO threshold.
- **Traffic:** Requests per second. Sudden drops can indicate outage.
- **Errors:** 5xx rate, exception rate. Alert if error rate > 1% for 5 minutes.
- **Saturation:** CPU, memory, disk, connection pool usage.

■ Alerting Best Practices

- **SLO-based alerts:** Alert when error budget burn rate is too high — not on raw metrics.
- Alert on symptoms (user impact), not causes (CPU is 80%) — reduces alert fatigue.
- **Multi-window alerting:** Short window catches fast burns, long window catches slow burns.
- **Runbook links in alerts:** Every alert should link to a runbook with debug steps.
- **PagerDuty/OpsGenie:** Route critical alerts to on-call. Silence non-actionable alerts.

■ Kubernetes-Specific

- **kube-state-metrics:** Exposes pod, deployment, node state as Prometheus metrics.
- **Node Exporter:** Hardware/OS metrics per node.
- **Prometheus Operator + ServiceMonitor:** Automatically discovers services to scrape.
- **Alertmanager:** Groups, routes, deduplicates, and silences alerts.

SECURITY

How do you implement secure authentication between services?

■ Option 1 — mTLS (Mutual TLS)

- Both client and server present certificates. Strongest form of service identity.
- Service meshes (Istio, Linkerd) handle mTLS automatically — no app code changes.
- Certificates auto-rotated, SPIFFE/SPIRE for identity.
- **Use when:** Zero-trust architecture, PCI/HIPAA environments.

■ Option 2 — JWT / OAuth2 Tokens

- Service A obtains a short-lived JWT from an auth server (Keycloak, AWS Cognito).
- Service B verifies JWT signature using public key — no auth server call per request.
- Scope/claim-based access control.
- **Use when:** External-facing APIs, user-context propagation across services.

■ Option 3 — AWS IAM + IRSA (Kubernetes)

- Pod assumes an IAM role via OIDC federation (IRSA — IAM Roles for Service Accounts).
- AWS services (S3, SQS, RDS IAM auth) verify the IAM role — no static credentials in pods.
- Best practice for AWS-native service-to-service auth.

■ Option 4 — API Keys / Shared Secrets (Avoid)

- Simple but risky — static keys, hard to rotate, no identity granularity.
- **If used:** Store in Vault/AWS Secrets Manager, rotate regularly, never in env vars or code.

■ Defence in Depth

- Network policy (Kubernetes NetworkPolicy) to restrict which pods can talk to which.
- Even with mTLS, apply least-privilege network rules.

DEPLOYMENT STRATEGIES

How do you design zero-downtime deployment strategies?

■ Rolling Update (Default in Kubernetes)

- Gradually replaces old pods with new ones. `maxSurge` controls extra pods, `maxUnavailable` controls how many can be down.
- Zero downtime if readiness probes are correctly configured — traffic only goes to ready pods.
- **Risk:** Both versions run simultaneously — ensure backwards-compatible API changes.

■ Blue-Green Deployment

- **Two identical environments:** Blue (live) and Green (new version).
- Green deployed and tested. Load balancer switched (Route53 weighted or ALB target group swap).
- **Instant rollback:** Switch back to Blue if issues arise.
- **Cost:** Requires 2x infrastructure during switch.
- **Use for:** High-risk releases, database schema changes (run both against same DB carefully).

■ Canary Deployment

- Route small % of traffic to new version (e.g., 5% → 10% → 25% → 100%).
- Monitor error rates and latency at each stage before promoting.
- **Kubernetes:** Argo Rollouts, Istio traffic splitting, ALB weighted target groups.
- **Best for:** Large user bases where you want to limit blast radius.

■ Key Enablers for Zero Downtime

- **readinessProbe:** Ensures traffic only goes to fully ready pods.
- **preStop hook + terminationGracePeriodSeconds:** Allow in-flight requests to complete before pod shutdown.
- **Backwards-compatible changes:** Never break API contracts in a single deploy — use versioning.
- **Database migrations:** Run before deploying new code. Use expand-contract pattern.

INCIDENT RESPONSE

If a production system goes down, how do you approach debugging step by step?

■ Immediate Response (0–5 min)

- 1. Acknowledge the alert. Notify the team / create incident channel.
- **2. Assess user impact:** Is it total outage or partial? Which regions/services?
- 3. Check recent deployments or config changes — most outages are change-induced.
- 4. Consider immediate rollback if a deploy happened in the last hour.

■ Diagnosis (5–20 min)

- **5. Check dashboards:** Error rate spike? Latency increase? Traffic drop?
- 6. `kubectl get pods -n production` — any `CrashLoopBackOff`, `Pending`, `OOMKilled`?
- 7. `kubectl logs --previous` — look for error messages.
- **8. Check infrastructure:** Are nodes healthy? Any cloud provider incidents (AWS Status page)?
- **9. Check database:** Connection pool exhausted? Replica lag? Disk full?
- **10. Trace the request path:** Load balancer → Service → Pod → Database. Where does it fail?

■ Mitigation

- 11. Prioritize restoring service over finding root cause — rollback first if needed.
- 12. Scale up if resource exhaustion — `kubectl scale` or ASG adjustment.
- **13. Feature flag:** Disable the broken feature if possible without full rollback.
- **14. Traffic redirect:** Point DNS/LB to backup region if regional failure.

■ Post-Incident

- **15. Write a blameless post-mortem:** Timeline, root cause, impact, contributing factors.
- **16. Define action items:** What monitoring would have caught this earlier? What automated remediation is needed?
- 17. Update runbooks. Conduct game day to test resilience.

■ Key Principle

- MTTR (Mean Time To Restore) matters more than MTBF (Mean Time Between Failures). Optimize for fast recovery.

You've Got This.

Prepare answers from your ApnaKart project and Hisan Labs experience.
Every question connects to something you've actually built.

AWS | Kubernetes | Terraform | Jenkins | Datadog | Docker

Good luck, Sanket.